

Data and Artificial Intelligence

Cyber Shujaa Program

Week 8 Assignment Classification Models

Student Name: Hope Njoki Nyambura

Student ID: cs-eh03-25100

Table of Contents

1. Introduction	1
2. Steps and Implementation.....	2
2.1 Loading Libraries and Dataset.....	2
2.2 Data Exploration and Visualization	2
2.3 Data Preparation	3
2.4 Building and Evaluating Classification Models.....	4
2.5 Model Comparison and Cross-Validation	5
3. Conclusion	5
4. Colab Notebook Link	6

1. Introduction

This week's assignment focused on implementing classification models using Python and Scikit-learn.

The goal was to understand how supervised machine learning algorithms can classify data into distinct categories based on input features.

The dataset used was the Wine dataset, which contains measurements of chemical properties in wines derived from three different grape cultivars.

The assignment involved:

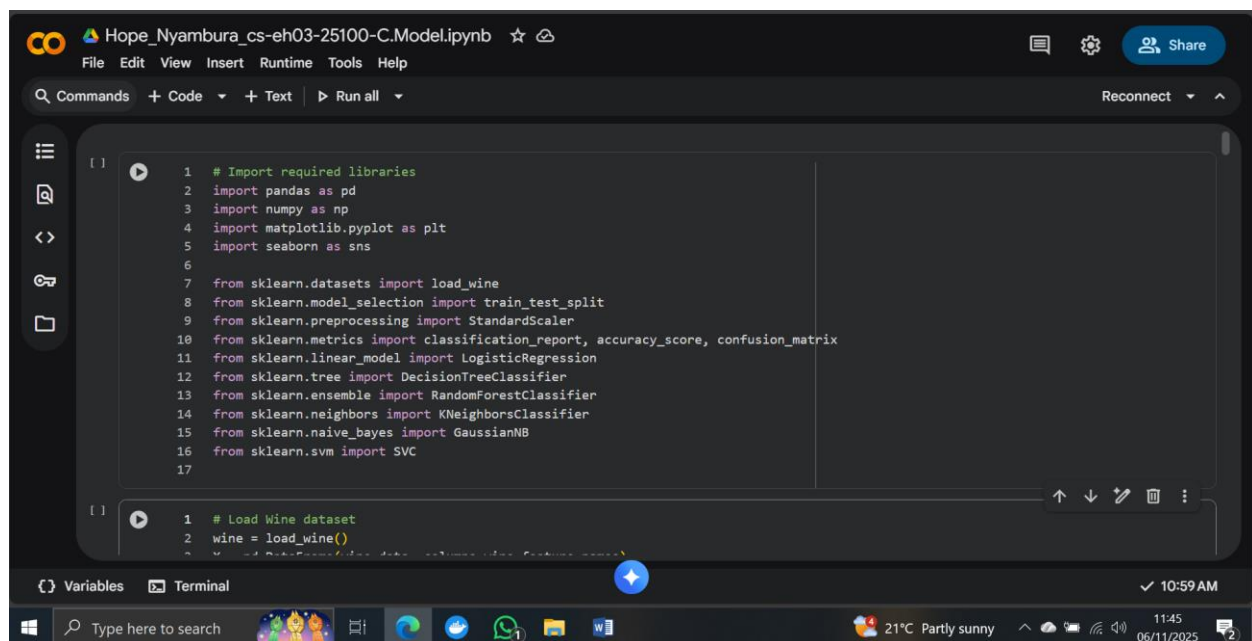
- Performing Exploratory Data Analysis (EDA) to understand the data distribution.
- Training six classification models — Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Naive Bayes, and Support Vector Machine (SVM).
- Evaluating and comparing model performance using accuracy scores, classification reports, and confusion matrices.

2. Steps and Implementation

2.1 Loading Libraries and Dataset

The first step involved importing key Python libraries including: pandas, numpy, matplotlib, seaborn, and Scikit-learn modules for data preprocessing, modeling, and evaluation.

The Wine dataset was loaded directly from `sklearn.datasets.load_wine()`. It contained 178 samples and 13 features, representing various chemical attributes of wine. Data exploration confirmed that there were no missing values, and each wine belonged to one of three classes.



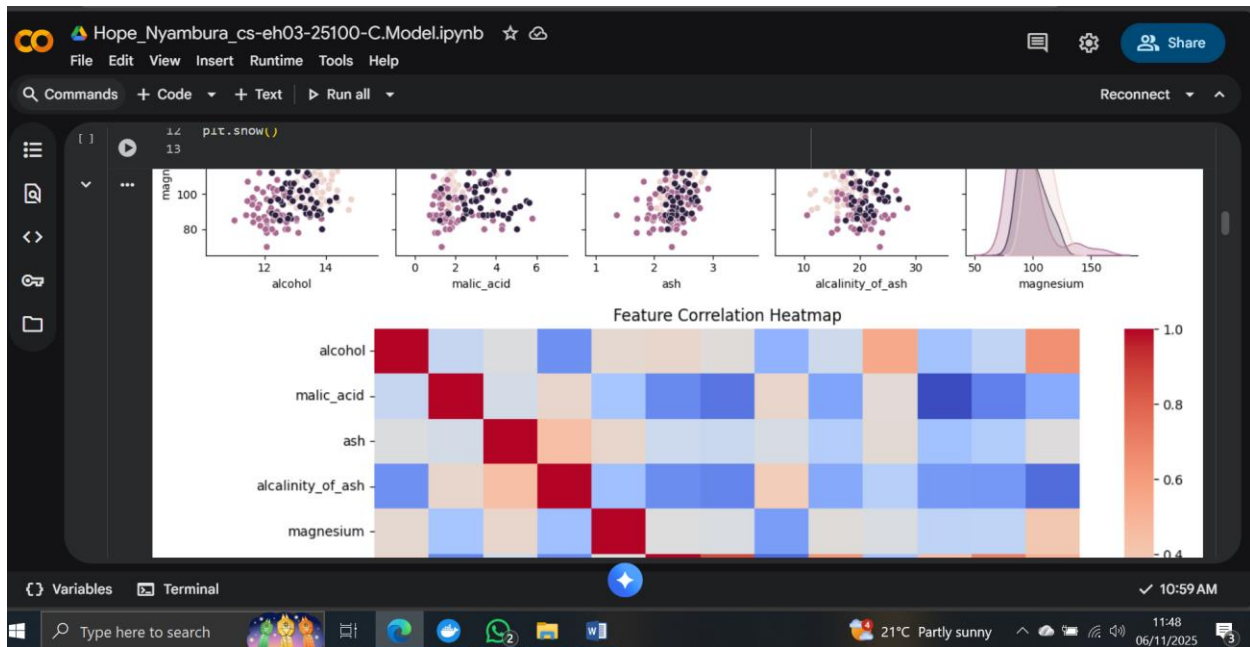
The screenshot displays a Jupyter Notebook window titled 'Hope_Nyambura_cs-eh03-25100-C.Model.ipynb'. The interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with icons for running code, saving, and other functions. The main area shows two code cells. The first cell contains a list of imports for pandas, numpy, matplotlib, seaborn, and various Scikit-learn modules. The second cell contains the code to load the Wine dataset using `load_wine()`. The bottom of the window shows a Windows taskbar with the search bar, task icons, and system tray information including the date and time.

```
[ ] 1 # Import required libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.datasets import load_wine
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
11 from sklearn.linear_model import LogisticRegression
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.ensemble import RandomForestClassifier
14 from sklearn.neighbors import KNeighborsClassifier
15 from sklearn.naive_bayes import GaussianNB
16 from sklearn.svm import SVC
17
[ ] 1 # Load Wine dataset
2 wine = load_wine()
```

2.2 Data Exploration and Visualization

Basic exploratory data analysis was conducted to understand feature relationships and class distributions.

- Descriptive statistics were generated to show feature ranges and variability.
 - Pairplots and correlation heatmaps were used to visualize how features differed among classes.
- These visualizations revealed strong correlations between certain chemical properties, suggesting possible feature importance for classification.



2.3 Data Preparation

Before training models, the features were standardized using `StandardScaler()` to ensure that all variables were on the same scale.

This step was crucial for models sensitive to feature magnitude such as KNN and SVM.

The data was then split into:

- 70% for training, and
- 30% for testing,
using Scikit-learn's `train_test_split()` with a random state of 42 for reproducibility.

The image shows a Jupyter Notebook interface with a file named 'Hope_Nyambura_cs-eh03-25100-C.Model.ipynb'. The notebook displays code for data preparation and a function for plotting a confusion matrix. The code includes the following lines:

```
1 X_scaled = StandardScaler().fit_transform(X)
2
3 # Train/Test Split (70% training, 30% testing)
4 X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)
5
6 print("Training set size:", X_train.shape)
7 print("Testing set size:", X_test.shape)
8
9
10
```

The output of the code shows the training set size as (124, 13) and the testing set size as (54, 13). Below the code, there is a function definition for plotting a confusion matrix:

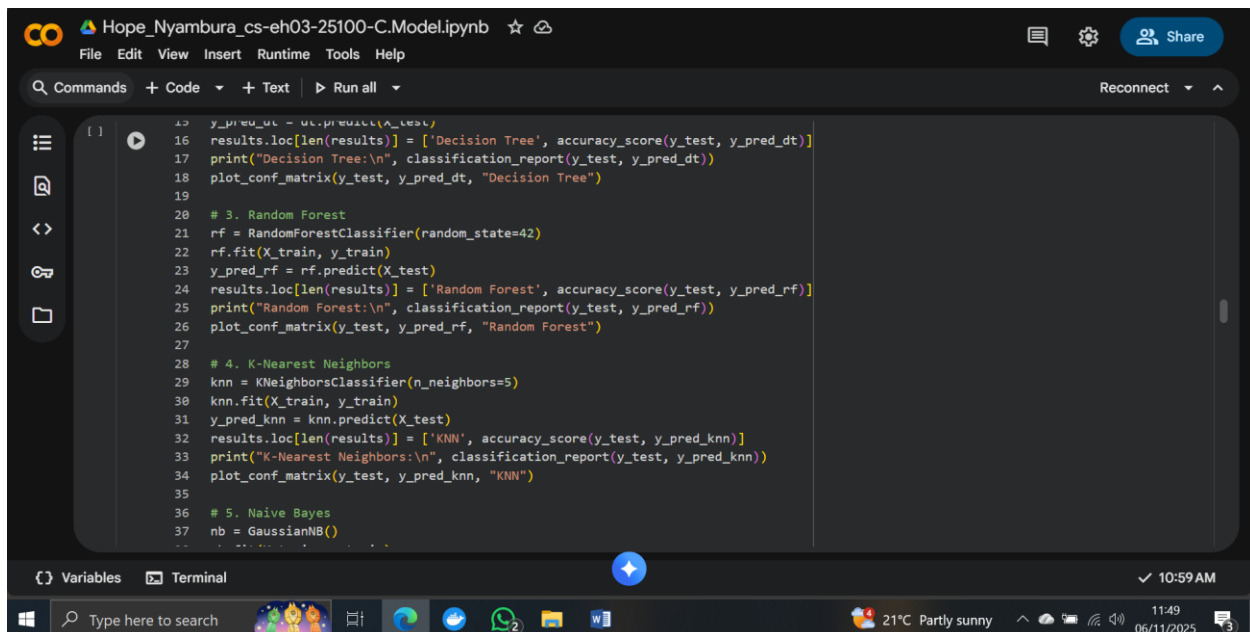
```
1 def plot_conf_matrix(y_true, y_pred, title):
2     cm = confusion_matrix(y_true, y_pred)
3     plt.figure(figsize=(5,4))
4     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
5                 xticklabels=wine.target_names, yticklabels=wine.target_names)
6     plt.xlabel('Predicted')
7     plt.ylabel('Actual')
8     plt.title(f'Confusion Matrix: {title}')
9     plt.show()
10
```

The Jupyter Notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a search bar, and a status bar at the bottom showing the time as 10:59 AM and the date as 06/11/2025.

2.4 Building and Evaluating Classification Models

Six models were trained and evaluated on the test set:

1. **Logistic Regression**
 - Achieved high accuracy and showed balanced precision and recall across all classes.
 - The confusion matrix indicated correct classification of most samples.
2. **Decision Tree Classifier**
 - Perfectly fit the training data but showed slight overfitting on the test set.
 - The confusion matrix showed some misclassifications between classes 1 and 2.
3. **Random Forest Classifier**
 - Provided strong performance with high overall accuracy and stable predictions.
 - The ensemble approach reduced overfitting compared to a single Decision Tree.
4. **K-Nearest Neighbors (KNN)**
 - Accuracy was good but slightly lower than Random Forest and SVM.
 - Model performance was sensitive to the chosen number of neighbors ($k=5$).
5. **Naive Bayes**
 - Performed decently despite its simplicity.
 - Assumed feature independence, which may not hold true for this dataset.
6. **Support Vector Machine (SVM)**
 - Produced one of the highest accuracy scores with the linear kernel.
 - Displayed a clean separation between wine classes.



```
Hope_Nyambura_cs-eh03-25100-C.Model.ipynb
File Edit View Insert Runtime Tools Help

Search Commands + Code + Text Run all Reconnect

15 y_pred_dt = dt.predict(X_test)
16 results.loc[len(results)] = ['Decision Tree', accuracy_score(y_test, y_pred_dt)]
17 print("Decision Tree:\n", classification_report(y_test, y_pred_dt))
18 plot_conf_matrix(y_test, y_pred_dt, "Decision Tree")
19
20 # 3. Random Forest
21 rf = RandomForestClassifier(random_state=42)
22 rf.fit(X_train, y_train)
23 y_pred_rf = rf.predict(X_test)
24 results.loc[len(results)] = ['Random Forest', accuracy_score(y_test, y_pred_rf)]
25 print("Random Forest:\n", classification_report(y_test, y_pred_rf))
26 plot_conf_matrix(y_test, y_pred_rf, "Random Forest")
27
28 # 4. K-Nearest Neighbors
29 knn = KNeighborsClassifier(n_neighbors=5)
30 knn.fit(X_train, y_train)
31 y_pred_knn = knn.predict(X_test)
32 results.loc[len(results)] = ['KNN', accuracy_score(y_test, y_pred_knn)]
33 print("K-Nearest Neighbors:\n", classification_report(y_test, y_pred_knn))
34 plot_conf_matrix(y_test, y_pred_knn, "KNN")
35
36 # 5. Naive Bayes
37 nb = GaussianNB()
38 nb.fit(X_train, y_train)
39 y_pred_nb = nb.predict(X_test)
40 results.loc[len(results)] = ['Naive Bayes', accuracy_score(y_test, y_pred_nb)]
41 print("Naive Bayes:\n", classification_report(y_test, y_pred_nb))
42 plot_conf_matrix(y_test, y_pred_nb, "Naive Bayes")

Variables Terminal 10:59 AM
Type here to search 21°C Partly sunny 11:49 06/11/2025
```

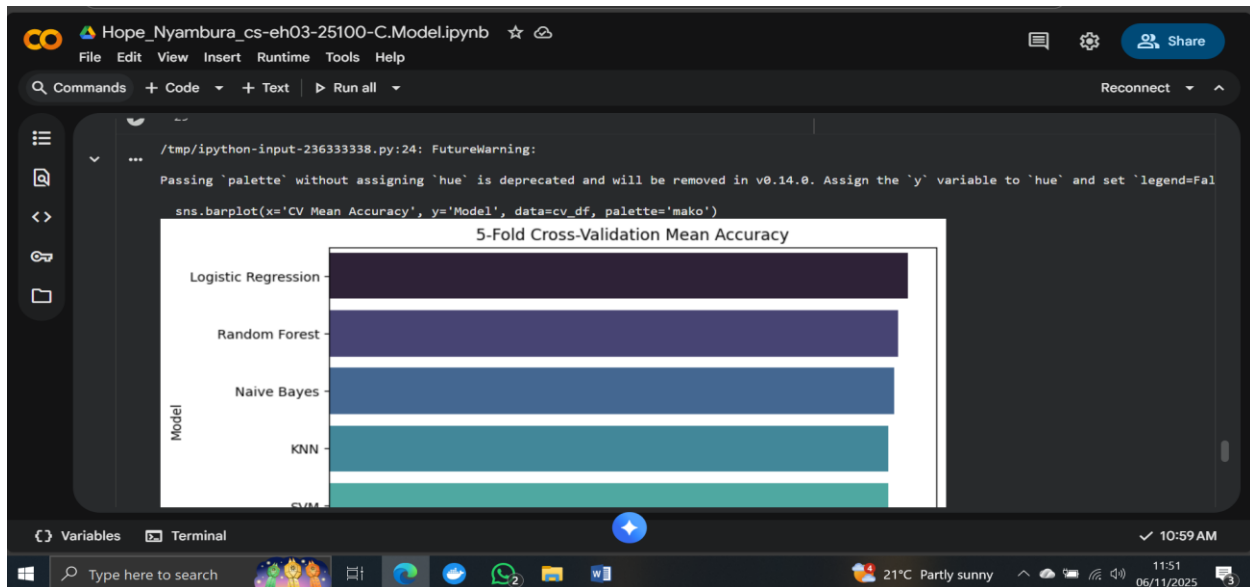
2.5 Model Comparison and Cross-Validation

All models were evaluated based on accuracy, precision, recall, and F1-score.

A bar chart comparison clearly showed Random Forest and SVM as the top-performing models.

To validate consistency, 5-fold cross-validation was applied.

The cross-validation results confirmed that SVM and Random Forest achieved the highest mean accuracy, demonstrating good generalization ability.



3. Conclusion

This assignment provided practical understanding of how various classification models work and how to evaluate their performance.

Through hands-on implementation, I learned how to:

- Load and preprocess real-world datasets.
- Apply and compare different supervised learning algorithms.
- Use performance metrics and confusion matrices to assess model accuracy.
- Implement cross-validation for more reliable evaluation.

Among all models, Support Vector Machine (SVM) and Random Forest performed the best overall due to their ability to handle complex relationships and avoid overfitting.

This assignment strengthened my skills in data analysis, model evaluation, and machine learning, which are essential for data-driven problem solving in cybersecurity and AI applications.

4. Colab Notebook Link

Google Colab Notebook:

[https://colab.research.google.com/drive/11Fidl_fcaxcRsiky0sapULAJ4889LviU?usp=sharing]